

Table of Contents

1. Introduction	1
Build with Dart everywhere	1
Who Serverpod is for	1
What's included	1
1.1. Installation	4
Prerequisites	4
Install Serverpod	4
Install the VS Code extension (recommended)	4
Install Serverpod Insights (optional)	4
1.2. Quickstart	7
Prerequisites	7
Install Serverpod	7
Create the project	7
Start the server and the app	7
Build your app	7
Deploy the app to the cloud	7
Get help	7
1.3. How it works	11
Create your project	11
Run your project	11
Write an endpoint	11
Call it from your app	11
Define your data models	11
Build with an AI agent	11
Next steps	11
1.4. Create your first endpoint	15
Before you start	15
Create the project	15
Start the app	15
Add an endpoint	15
Call it from your app	15
Next steps	15

1.5. Create data models	21
Define a model	21
Return the model from your endpoint	21
Show it in your app	21
Next steps	21
1.6. Manage the database	24
Map the model to a table	24
Create and apply the migration	24
Save recipes to the database	24
List past recipes	24
Show the saved recipes in your app	24
Next steps	24
1.7. Deploy your app	30
Deploy to Serverpod Cloud	30
Other deployment options	30
What you've built	30
2. Serverpod Academy	32
2.1. Backend fundamentals	33
2.2. Real-time communication	34
What is Pixorama?	34
Understanding real-time communication	34
Setting up the project	34
Creating models	34
Building the server	34
Building the Flutter app	34
Running Pixorama	34
Conclusion	34
2.3. Integrating AI & RAG	45
3. Support & community	46
Reporting issues	46
Stay up-to-date	46
Community updates	46
3..1. Your Serverpod project	48
How the packages relate	48
Where your code lives, and what is generated	48
The server entry point	48

The three servers	48
The config and migrations directories	48
Editor and agent files	48
Related	48
3..2. Running your server	52
Save a file to hot reload	52
Manage migrations from the terminal	52
Choose a run mode	52
Choose which Flutter apps start	52
Run the server on its own	52
Run the server directly	52
Choose a server role	52
Run code on shutdown	52
Related	52
3..3. Configuration	57
How configuration works	57
Run modes	57
Configure with YAML files	57
Configure in Dart	57
Manage secrets	57
Configure code generation	57
Related	57
3..4. Modules	67
Add a module to your project	67
Reference a module in your models	67
Create your own module	67
Related	67
3..5. Working with endpoints	71
Create an endpoint	71
Call an endpoint from your app	71
Point the client at each environment	71
Restrict access to your endpoints	71
Pass and return data	71
Exclude an endpoint from generation	71
Test your endpoints	71
Related	71

3.6. Sessions	76
Session types	76
What a session provides	76
Session lifecycle	76
Create a session for background work	76
Run work after the call returns	76
Logging	76
Test with sessions	76
Related	76
3.7. Error handling and exceptions	81
Serializable exceptions	81
Handle errors in your app	81
Don't leak sensitive data	81
Related	81
3.8. Streaming	87
Streaming methods	87
Example: live updates for a filtered query	87
3.9. Server events	92
Send messages	92
Receive messages	92
Broadcast revoked authentication	92
3.10. Caching	95
The caches	95
Cache an object	95
Invalidate entries	95
The global cache and Redis	95
Send Redis commands directly	95
Related	95
3.11. File uploads	100
Upload a file	100
Size limits	100
Access stored files	100
Configure a storage provider	100
Use a custom S3-compatible endpoint	100
Related	100
3.12. Custom cloud storage	110

When to use a custom backend	110
How uploads reach storage	110
Store files on local disk	110
Register the storage	110
Raise the request size limit	110
Authorize the upload endpoints	110
Implement the CloudStorage methods	110
Use an S3-compatible store	110
Troubleshooting	110
Related	110
3..13. Endpoint inheritance	119
Inheriting from an Endpoint class	119
Inheriting from an Endpoint class marked abstract	119
Inheriting from an Endpoint class annotated with @doNotGenerate	119
Overriding endpoint methods	119
Hiding endpoint methods with @doNotGenerate	119
Unhiding endpoint methods annotated with @doNotGenerate in the super class	119
Building base endpoints for behavior	119
Client-side endpoint inheritance	119
3..14. Backward compatibility	128
Managing breaking changes with endpoint inheritance	128
3..15. Endpoint middleware	130
Add middleware to your server	130
Create custom middleware	130
Handle errors in middleware	130
3..16. Configure HTTP calls	134
Include CORS credentials on web	134
Use platform-native HTTP clients	134
3..16.1. Working with models	138
Class	138
Default values	138
Exception	138
Enum	138
Adding documentation	138
Generated code	138
Related	138

3..16.2. Field types	151
Overview	151
Nullability	151
Serialization	151
Primitive types	151
Enums	151
Collections and records	151
Nested models	151
JSON vs JSONB	151
Vector types	151
Geography types	151
Dynamic and custom classes	151
ID types	151
Defaults	151
Indexes	151
Query operators	151
Postgres vs SQLite	151
Related	151
3..16.3. Inheritance and polymorphism	164
Inheritance	164
Polymorphism	164
3..16.4. Dynamic fields	171
Define a dynamic field	171
Supported values	171
Use dynamic values in endpoints	171
Store dynamic fields in the database	171
Copy models with dynamic fields	171
Limits	171
Related	171
3..16.5. Custom serialization	175
Using a custom class as a model field	175
Where custom classes live	175
Custom class with Freezed	175
Custom class with ProtocolSerialization	175
3..16.6. Shared packages	179
Setup	179

Using shared models	179
Shared packages in a module	179
Restrictions	179
3..16.7. Tables	184
Choosing where a table lives	184
Non persistent fields	184
Data representation	184
Choosing an ID strategy	184
Change ID type	184
Column name override	184
3..16.8. CRUD	192
Create	192
Read	192
Update	192
Upsert	192
Delete	192
Skipping returned rows	192
Count	192
3..16.9. Filter	201
Column operations	201
Logical operators	201
Vector distance operators	201
Relation operations	201
Geography operators	201
3..16.10. Sort	212
Sort on relations	212
3..16.11. Pagination	214
Limit	214
Offset	214
Using limit and offset for pagination	214
Cursor-based pagination	214
3..16.12. Relations	217
Choose a relation shape	217
Related	217
3..16.13. Relation queries (Joins)	219
Include relational data	219

Include relational lists	219
Attach and detach relations	219
3..16.14. Vector & geography fields	224
Vector fields	224
Geography fields	224
3..16.15. Indexing	229
Add an index	229
Make fields unique	229
Specify an index type	229
GIN indexes	229
Vector indexes	229
Geography indexes	229
3..16.16. Transactions	237
Cancel a transaction	237
Transaction isolation	237
Deferred constraint checking	237
Transaction failure exceptions	237
Savepoints	237
3..16.17. Row locking	242
Locking rows with a read	242
Locking rows without fetching data	242
Lock modes	242
Lock behavior	242
3..16.18. Raw access	246
unsafeQuery	246
unsafeExecute	246
unsafeSimpleQuery	246
unsafeSimpleExecute	246
Query parameters	246
Working with results	246
Intercept database access	246
3..16.19. Migrations	250
Opt out of migrations	250
Creating a migration	250
Apply migrations	250
Creating a repair migration	250

Applying a repair migration	250
Rolling back migrations	250
Troubleshooting	250
3..16.20. Connection	258
Connection details	258
Database password	258
The development database	258
Connect to a custom Postgres instance	258
3..16.21. Runtime parameters	262
Parameter scopes	262
Parameter groups	262
Database schema search paths	262
Vector runtime parameters	262
Testing with runtime parameters	262
3..16.22. Client-side database	271
Enable the client-side database	271
Create a session	271
Run migrations on the device	271
Use the database	271
Sync tables with the server	271
3..16.23. Database exceptions	276
Exception types	276
Query exception details	276
Operation exceptions	276
SQLite foreign key checks	276
3..16.24. Database interceptors	279
Return a custom database layer	279
3..16.25. Embedded PostgreSQL	281
Enable it	281
What happens when the server starts	281
Connect a database tool	281
Run integration tests	281
Reset the database	281
Related	281
3..17. Get started	284
Prerequisites	284

Show the sign-in screen	284
Start your project	284
Next steps	284
3..18. Setup	288
Server setup	288
Client setup	288
App setup	288
Related	288
3..19. The basics	299
Requiring authentication on endpoints	299
Explicitly allowing unauthenticated access	299
Authorization on endpoints	299
Client-side authentication	299
Signing out users	299
Related	299
3..20. Working with users	307
Authenticated users	307
Blocking users	307
User creation callbacks	307
User profiles	307
Attaching additional information	307
Merging accounts	307
Related	307
3..20..1. Setup	316
Server-side configuration	316
Client-side configuration	316
3..20..2. Customizations	319
Server configuration	319
App configuration	319
Customize the sign-in button	319
Build a custom UI with AnonymousAuthController	319
Related	319
3..20..3. Setup	325
Server-side configuration	325
Client-side configuration	325
3..20..4. Configuration	329

Configuration options	329
3..20..5. Customizing the UI	333
Using the EmailSignInWidget	333
Building a custom UI with the EmailAuthController	333
The email authentication flow	333
3..20..6. Admin operations	339
Accessing the EmailIdpAdmin	339
Account management	339
Finding active account requests	339
Cleanup operations	339
3..20..7. Setup	343
Prerequisites	343
Get your Google credentials	343
Server-side configuration	343
Client-side configuration	343
Present the authentication UI	343
Publishing to production	343
3..20..8. Customizations	361
Server configuration	361
App configuration	361
Customize the sign-in button	361
Build a custom UI with GoogleAuthController	361
Related	361
3..20..9. Admin operations	371
Accessing the GoogleIdpAdmin	371
Finding accounts	371
Linking Google authentication	371
Related	371
3..20..10. Troubleshooting	373
Setup checklist	373
Sign-in fails with redirect_uri_mismatch	373
Production redirect URIs rejected by Google	373
Sign-in works for you but not for other users	373
Sign-in callback fails locally with flutter run -d chrome	373
Sign-in callback never returns to the Flutter app	373
Server fails to parse googleClientSecret from passwords.yaml	373

Sign-in fails on Android with PlatformException(sign_in_failed) or clientConfig...	373
Sign-in works in debug but fails in release	373
Sign-in fails on Android with "serverClientId must be provided"	373
Endpoint calls fail on Android with connection refused	373
Server crashes on first Google sign-in with "no such table"	373
Google sign-in button does not appear	373
Lightweight sign-in (One Tap) not appearing	373
iOS sign-in prompt doesn't show	373
clientId or redirectUri is required when initializing Google Sign-In on web	373
Google API calls fail after one hour	373
Related	373
3..20..11. Setup	382
Prerequisites	382
Get your credentials	382
Server-side configuration	382
Client-side configuration	382
Present the authentication UI	382
Publishing to production	382
3..20..12. Customizations	395
Server configuration	395
App configuration	395
Customize the sign-in button	395
Build a custom UI with AppleAuthController	395
Related	395
3..20..13. Admin operations	403
Accessing the AppleIdpAdmin	403
Checking account status	403
Related	403
3..20..14. Troubleshooting	405
Setup checklist	405
Sign-in fails with invalid_client every time	405
Sign-in starts failing with invalid_client after months of success	405
Sign-in fails with invalid_grant	405
Wrong identifier passed for web or Android sign-in	405
Sign-in hangs on Android	405
Server crashes on first Apple sign-in with "no such table"	405

Apple rejects the redirect URI with invalid_request	405
Emails aren't delivered to some users	405
User email is null after sign-in	405
iOS sign-in prompt doesn't show	405
Sign-in does not work in the iOS Simulator	405
Web sign-in fails with TypeError: type ... is not a subtype of type 'JSObject'	405
macOS sign-in shows "Sign Up Not Completed"	405
User stays signed in after removing Apple access	405
Related	405
3..20..15. Setup	412
Prerequisites	412
Create your Facebook app	412
Server-side configuration	412
Client-side configuration	412
Present the authentication UI	412
Publishing to production	412
3..20..16. Customizations	430
Server configuration	430
App configuration	430
Customize the sign-in button	430
Build a custom UI with FacebookAuthController	430
Related	430
3..20..17. Troubleshooting	439
Setup checklist	439
Web sign-in fails with "JSSDK Option is Not Toggled"	439
Web sign-in is blocked by CORS or the domain is not allowed	439
iOS sign-in fails after denying tracking permission	439
Android opens Facebook but the sign-in never completes	439
Sign-in works for you but not for other users	439
Sign-in succeeds but the user has no email	439
Server crashes on startup with a missing password	439
Server crashes on first Facebook sign-in with "no such table"	439
Sign-in works in dev but fails after deploy	439
Related	439
3..20..18. Setup	444
Prerequisites	444

Get your credentials	444
Server-side configuration	444
Client-side configuration	444
Present the authentication UI	444
Publishing to production	444
3..20..19. Configuration	457
Loading credentials with FirebaseAuthConfig	457
Custom account validation	457
Reacting to auth user creation	457
FirebaseAuthConfig parameter reference	457
3..20..20. Customizing the UI	462
Anatomy of the gate widget	462
Controller reference	462
Using firebase_auth directly	462
3..20..21. Admin operations	467
Accessing the FirebaseAuthAdmin	467
Finding accounts	467
Linking Firebase authentication	467
Deleting accounts	467
FirebaseAuthUtils	467
3..20..22. Troubleshooting	470
Setup checklist	470
Server crashes on first Firebase sign-in with "no such table"	470
Token verification fails with "invalid signature"	470
Token verification fails with "token expired"	470
Server fails to parse firebaseServiceAccountKey from passwords.yaml	470
FirebaseAuth.instance.currentUser is null after sign-in	470
Sign-in succeeds in Firebase but fails to sync with Serverpod	470
Email validation rejects phone-only users	470
User signed out of Serverpod but still signed in to Firebase	470
FlutterFire configure fails or generates wrong config	470
FlutterFire configure fails for iOS or macOS apps	470
Firebase UI auth actions not firing	470
Platform-specific Firebase SDK configuration issues	470
Firebase.initializeApp() throws channel-error on web in production	470
3..20..23. Setup	477

Prerequisites	477
Get your GitHub credentials	477
Server-side configuration	477
Client-side configuration	477
Present the authentication UI	477
Publishing to production	477
3..20..24. Customizations	488
Server configuration	488
App configuration	488
Customize the sign-in button	488
Build a custom UI with GitHubAuthController	488
Related	488
3..20..25. Troubleshooting	498
Setup checklist	498
Sign-in fails with redirect_uri_mismatch	498
Callback never returns to the Flutter app	498
Sign-in succeeds but the user has no email	498
clientId or redirectUri missing at initialization	498
Sign-in works in dev but fails after deploy	498
Sign-in works for you but not for other users	498
Organization blocks the OAuth authorization	498
Sign-in fails with bad_verification_code	498
GitHub API calls from getExtraGitHubInfoCallback fail or rate-limit	498
Permission changes on the GitHub App do not take effect	498
Server crashes on startup with a missing password	498
Server crashes on first GitHub sign-in with "no such table"	498
Android sign-in opens GitHub but the callback never fires	498
Related	498
3..20..26. Setup	505
Create your Microsoft Entra ID app	505
Get app credentials	505
Server-side configuration	505
Client-side configuration	505
Present the authentication UI	505
3..20..27. Customizations	516
Server configuration	516

App configuration	516
Customize the sign-in button	516
Build a custom UI with MicrosoftAuthController	516
Related	516
3..20..28. Troubleshooting	525
Setup checklist	525
Sign-in fails with a redirect URI error	525
Callback never returns to the Flutter app	525
Sign-in fails with an access token verification error	525
clientId or redirectUri missing at initialization	525
The sign-in button does nothing and onError never fires	525
Changed initialization values do not take effect	525
Sign-in breaks after changing the scopes parameter	525
Users see a "Need admin approval" screen	525
Some account types cannot sign in	525
Sign-in works on mobile but fails on web, or the reverse	525
Sign-in succeeds but the user has no email, or the wrong one	525
Users have no profile photo	525
Calls from getExtraMicrosoftInfoCallback fail or block sign-in	525
Sign-in works in dev but fails after deploy	525
Server fails to start with a missing password error	525
Server crashes on first Microsoft sign-in with "no such table"	525
Android sign-in opens Microsoft but the callback never fires	525
Related	525
3..20..29. Setup	534
Server-side configuration	534
Client-side implementation	534
3..20..30. Custom providers	537
Related	537
3..20..30.1. Setup	539
Understanding OAuth2 with PKCE	539
Server-side implementation	539
Client-side implementation	539
Complete example of a custom provider	539
Best practices	539
3..20..30.2. Create an OAuth2-based identity provider	547

Overview	547
Server-side implementation	547
Client-side implementation	547
3..20.1. Managing tokens	561
Default token managers	561
Using the token managers	561
Token lifecycle management	561
Accessing specific token managers	561
Related	561
3..20.2. JWT token manager	566
Server-side configuration	566
Client-side configuration	566
Related	566
3..20.3. Server-side sessions token manager	572
Server-side configuration	572
Client-side configuration	572
Related	572
3..21. UI components	576
SignInWidget	576
Localization	576
Related	576
3..22. Profile photos	581
Before you start	581
Expose the profile edit endpoint	581
Upload a profile photo from Flutter	581
Display the profile photo	581
Remove or replace a photo	581
Configure image size and format	581
Configure storage for production	581
Use server-side APIs for custom logic	581
Verify	581
Troubleshooting	581
Related	581
3..23. macOS setup	589
Before you start	589
Add the Keychain Sharing entitlement	589

Verify	589
Advanced	589
Troubleshooting	589
Related	589
3..24. Web setup	593
Before you start	593
Configure the server	593
Configure the client	593
Verify	593
How it works	593
Cross-site request protection	593
Cross-subdomain cookies	593
Local development	593
Related	593
3..25. Custom overrides	597
Server setup	597
Client setup	597
Authentication schemes	597
Related	597
3..25.1. Setup	603
Installing the auth module	603
Server setup	603
Client setup	603
App setup	603
3..25.2. The basics	612
Requiring authentication on endpoints	612
Explicitly allowing unauthenticated access	612
Authorization on endpoints	612
User authentication	612
3..25.3. Working with users	617
Displaying or editing user images	617
3..25.3.1. Email	619
Server-side configuration	619
Client-side configuration	619
Custom UI with EmailAuthController	619
Password storage security	619

Custom password hash generator	619
3..25.3.2. Google	625
Create your credentials	625
Server-side configuration	625
Client-side configuration	625
Calling Google APIs	625
3..25.3.3. Apple	633
Server-side configuration	633
Client-side configuration	633
Extra	633
3..25.3.4. Firebase	637
Server-side configuration	637
Client-side configuration	637
Firebase config	637
Trigger the auth UI with Serverpod	637
3..25.3.5. Custom providers	641
Server setup	641
Client setup	641
3..25.4. Custom overrides	648
Server setup	648
Client setup	648
Authentication schemes	648
3..26. Overview	655
Your first route	655
Core concepts	655
When to use what	655
Database access	655
Going to production	655
Next steps	655
3..27. Routing	660
Route classes	660
How requests are matched	660
HTTP methods	660
Path parameters	660
Wildcards	660
Fallback routes	660

Virtual host routing	660
Next steps	660
3..28. Request data	667
Path parameters	667
Query parameters	667
Headers	667
Body	667
Next steps	667
3..29. Web server middleware	671
Adding middleware	671
Creating custom middleware	671
Middleware execution order	671
Request-scoped data	671
Next steps	671
3..30. Static files	676
Serving static files	676
Cache control	676
Static file cache-busting	676
Conditional requests (ETags and Last-Modified)	676
3..31. Server-side HTML	680
Creating a WidgetRoute	680
Creating a TemplateWidget	680
Built-in widgets	680
Database access and logging	680
Rendering with Jaspr	680
Next steps	680
3..32. Single-page apps	685
Basic setup	685
How it works	685
Cache control	685
Cache busting	685
Using FallbackMiddleware directly	685
Serving from a sub-path	685
Serving Flutter web applications	685
3..33. Flutter web apps	689
Basic setup	689

Building Flutter for web	689
Cache control	689
WASM builds	689
Complete example	689
3..34. Overview	694
What you can schedule	694
Execution guarantees	694
Where future calls run	694
What each page covers	694
Related	694
3..35. Future calls	696
Define a future call	696
Schedule a call	696
Cancel scheduled calls	696
Inspect scheduled calls	696
Execution timeout	696
Related	696
3..36. Recurring tasks	700
Repeat on a fixed interval	700
Repeat on a cron schedule	700
Stop a recurring task	700
Failure behavior	700
Reschedule manually	700
Related	700
3..37. Inheritance	703
Extend a class	703
Expose future calls from a module	703
Override an inherited method	703
Related	703
3..38. Configuration	706
Execution options	706
Broken future calls	706
Related	706
3..39. Overview	710
What each page covers	710
Related	710

3.40. Logging	712
Write your own messages	712
Where logs go	712
Which sessions get recorded	712
Configure logging	712
Purge old records	712
Related	712
3.41. Health checks	716
Health probes	716
Health metrics	716
Related	716
3.42. Security and TLS	721
Serve HTTPS from the server	721
Trust the server's certificate from your app	721
Related	721
3.43. Exception monitoring	723
Add a handler	723
Report an exception yourself	723
Related	723
3.44. Scalability	725
How Serverpod scales	725
Split request and maintenance work	725
Offload CPU work to isolates	725
Optimize Postgres queries	725
Size the connection pool	725
Scale Postgres itself	725
Prefer JWT for auth at scale	725
Share state across instances with Redis	725
Choose streams carefully	725
Configuration cheat sheet	725
Limits	725
Related	725
3.45. Redis	730
Connection details	730
Redis password	730
Enable Redis	730

The development Redis	730
Connect a GUI	730
Test, staging, and production	730
SSL and username	730
What Redis is used for	730
Troubleshooting	730
Related	730
3..46. Load testing	738
Before you start	738
Run a production-like server	738
Call an endpoint over HTTP	738
Write a Locust scenario	738
Measure the server	738
Check these settings before you run	738
Drive load with the generated client	738
Scenario types	738
Troubleshooting	738
Related	738
3..47. Get started	744
Set up	744
Write a test	744
Next	744
3..48. Writing tests	746
Set up a scenario	746
Simulate authentication	746
Seed the database	746
Exceptions the test tools throw	746
Wait for async work	746
Related	746
3..49. Configuration	750
Options	750
Run mode	750
Database rollback	750
Server output	750
Test tags	750
Related	750

3..50. Advanced examples	755
Run unit and integration tests separately	755
Test business logic that depends on Session	755
Multiple users interacting with a shared stream	755
Run a future call	755
Too many database connections	755
Testing exception monitoring	755
Related	755
3..51. Best practices	760
Imports	760
Database clean up	760
Calling endpoints	760
Unit and integration tests	760
Related	760
3..52. Introduction	763
Install the CLI	763
Command syntax	763
Global options	763
Environment variables	763
How this reference is organized	763
3..53. Global Options	765
Usage	765
3..54. Environment Variables	767
3..54.1. cloud	768
3..54.2. completion	769
Usage	769
3..54.3. create	771
Usage	771
3..54.4. create-migration	773
Usage	773
3..54.5. create-repair-migration	774
Usage	774
3..54.6. database	775
Usage	775
3..54.7. generate	777
Usage	777

3..54.8. language-server	778
Usage	778
3..54.9. mcp-server	779
Usage	779
3..54.10. quickstart	780
Usage	780
3..54.11. run	782
Usage	782
3..54.12. start	783
Usage	783
3..54.13. upgrade	785
Usage	785
3..54.14. version	786
Usage	786
3..55. Configuration reference	787
Run options	787
Server and services	787
Code generation	787
Related	787
3..56. Model reference	798
Related	798
3.1. Deploy to Serverpod Cloud	802
Deploy from the terminal	802
3.1.1. Choosing a strategy	804
Server cluster or serverless?	804
Pick a guide	804
Approximate costs	804
3.1.2. Hosting elsewhere	806
Required services	806
Configuration files	806
Server roles	806
Docker container	806
3.1.3. Community-supported deployments	809
Virtual Private Server (VPS)	809
3.2. Serverpod Insights	810
Downloads	810

Database access	810
3.3. LSP server	812
Navigate from Dart code	812
3.4. Run scripts	814
3.5. Command line completion	816
Installing completion	816
Alternative: Completely	816
3.6. Upgrade to 4.0	819
Before you start	819
Update the Serverpod CLI	819
Update your project dependencies	819
Other breaking changes	819
Authentication changes	819
Generate the 4.0 migration	819
Production deployment notes	819
Adopt the new development workflow (optional)	819
Set up the agent workflow (optional)	819
What's new in 4.0	819
Troubleshooting	819
Still stuck?	819
Related	819
3.6.1. Upgrade to 1.2	838
Updating your CLI	838
Updating your pubspecs	838
Deprecated methods	838
Model changes	838
Moved and renamed SQL file	838
Initialize the migration system	838
Closing remarks	838
3.6.2. Upgrade to 2.0	845
Changes to authentication	845
Changes to the Session Object	845
Changes to database queries	845
Changes to database tables	845
Changes in the authentication module	845
Updates to Serialization in Serverpod 2.0	845

Deprecation Notice for SerializableEntity	845
3.6.3. Upgrade from Mini to full	855
3.6.4. Upgrade to 2.2	856
Add test tools to existing projects	856
3.6.5. Upgrade to pgvector	859
For Docker-based environments	859
For managed PostgreSQL services	859
Troubleshooting	859
3.6.6. Upgrade to 3.0	862
Requirements	862
Breaking changes summary	862
Updating your project	862
Migration checklist	862
Web server (Relic framework)	862
Session.request property	862
Authentication	862
Enum serialization	862
Model changes	862
Deprecated APIs	862
Other changes	862
Resources	862
3.6.7. Streaming endpoints (removed in 4.0)	872
Handling streams server-side	872
Handling streams in your app	872
3.7. Migrate from legacy auth	874
Before you start	874
Add the new auth packages	874
Configure the server	874
Run the migration	874
Wire up sign-in for migrated users	874
Update the Flutter app	874
Verify and clean up	874
Troubleshooting	874
Still stuck?	874
Related	874
3.8. Upgrading to PostGIS support	884

For Docker-based environments	884
For managed PostgreSQL services	884
Troubleshooting	884
4. Roadmap & contributions	887
Roadmap	887
Contributions	887
 APPENDIX	
<i>Appendix A. Deferrable constraints Serverpod</i>	<i>889</i>
Declare a deferrable relation	889
Defer the check in a transaction	889
Related	889
<i>Appendix B. Many-to-many Serverpod</i>	<i>892</i>
When to use a junction table	892
Defining the relationship	892
Managing junction records	892
Common errors	892
<i>Appendix C. Relations with modules Serverpod</i>	<i>895</i>
Advanced example	895
<i>Appendix D. One-to-many Serverpod</i>	<i>897</i>
Defining the relationship	897
Bidirectional relations	897
<i>Appendix E. One-to-one Serverpod</i>	<i>900</i>
Defining the relationship	900
Independent relations defined on both sides	900
Bidirectional relations	900
<i>Appendix F. Referential actions Serverpod</i>	<i>906</i>
Available referential actions	906
Syntax	906
Default values	906
<i>Appendix G. Self-relations Serverpod</i>	<i>908</i>
One-to-one	908
One-to-many	908
Many-to-many	908